

KopaAlert

A record of how it was built, why it took the shape it did, and what comes after this document closes.

PREPARED BY	Stanley Murangiri M'mwirambua
ROLE	Founder & Sole Developer — B.Sc. Computer Science, Mama Ngina University
COMPANY	Solution Tech Company Ltd (pre-incorporation)
LIVE PRODUCT	<code>kopaalert.shop</code>
ISSUED	September 2026

KopaAlert is a debt-tracking and automated reminder platform for Kenyan businesses that sell on credit. This dossier is written for three readers at once — a business owner deciding whether to trust it with their customers, a developer weighing whether to build on it, and the record its author keeps of the first real product built under his own name.

WHY THIS EXISTS

Selling on credit shouldn't mean losing track of who owes what.

Across Kenya, small and medium businesses routinely extend credit to trusted customers — a practice known locally as *kopa*. It runs on notebooks, memory, and the owner's willingness to make an awkward phone call when a payment is late.

KopaAlert replaces the notebook. A business registers, is reviewed and approved, and then records every customer it extends credit to, along with the amount, the due date, and every payment made against it. From there, the system takes over the part that used to depend on memory: it watches due dates and sends the reminder — by SMS and email — automatically, on the business's behalf, without the owner having to ask.

What the platform actually does

Once a business is live on KopaAlert, day-to-day debt tracking becomes a dashboard instead of a notebook: customers and debts are recorded once, reminders go out on schedule, payments are logged as they come in, overdue accounts are flagged automatically, and the owner can see the whole picture — who owes what, who's paid, who's gone quiet — at a glance.

The messaging itself runs on shared infrastructure that KopaAlert operates centrally — one SMS gateway account, one email sender — so an individual business never has to negotiate its own contract with a telecom or configure its own mail server just to remind a customer that a payment is due.

Everything past this page is written to be read on its own: skip ahead by role using the contents on the left, or read straight through as one document.

WHO BUILT THIS

Stanley Murangiri M'mwirambua

Computer Science student, Mama Ngina University — and the sole developer behind everything described in this document.

Stanley studies Computer Science with a particular pull toward the parts of the field where problem-solving has a visible, immediate payoff: a real workflow that gets easier for a real person once the code is running. KopaAlert grew directly out of that instinct — not an assignment, not a tutorial project, but a system built because a genuine gap (Kenyan traders tracking credit informally, with no tooling built for how they actually work) was worth closing.

That same instinct is what led him to start thinking beyond a single project toward a company of his own: **Solution Tech Company Ltd.** It isn't incorporated yet — that step is still ahead — but it is already the name under which KopaAlert was designed, built, and is now being documented, and the entity he intends future products to sit under as well.

KopaAlert is, in that sense, the first proof of the idea: that one developer, working alone, can take a product from an empty repository to a live, paying, multi-tenant platform — schema design, backend, frontend, billing, and the SMS/email integrations that make the reminders real — without a team, a funding round, or someone else's roadmap to follow.

BUILT AGAINST FRICTION

What made this slower than it should have been.

None of this is offered as an excuse for what isn't finished — it's the honest record of what building KopaAlert alone, without outside funding, actually involved.

01 Capital

Almost every piece of real infrastructure charges by the month once you outgrow the free tier: Vercel for hosting, GitHub for source control at scale, Cloudflare for domain and DNS protection, Supabase for the database and auth layer. Even the domain itself — `kopaalert.shop` — was a real cost, about KES 500, paid out of pocket. Funding this without outside investment has been the single biggest reason KopaAlert took this long to reach real, paying customers.

02 Time

KopaAlert was built around an already-full daily routine, not instead of it. Most of the actual code was written at night, after everything else that day required was done.

03 Finding real businesses

Writing the software turned out to be the easier half. Finding actual Kenyan businesses willing to hand a new, unproven system their customer and payment data — and trust it over the notebook they already know — has been the slower problem, and it's still an open one.

04 Electricity and connectivity

Power and internet reliability are not guaranteed. Development sessions and testing runs have repeatedly been cut short or delayed by outages outside anyone's control.

05 Hardware

The entire platform — every line described in this dossier — has been built on a laptop that is overdue for an upgrade. A meaningful share of the friction above is simply that: the tool doing the job is not the right size for the job anymore.

06 The product itself kept changing

What KopaAlert is today is not what it set out to be. Earlier ideas were tried, didn't hold up once real users or real constraints met them, and were rebuilt. That's presented here plainly, not as a failure to plan correctly, but as the ordinary cost of building something real without a team or someone else's finished roadmap to follow.

Every one of these is still true today. This dossier documents where the project stands in spite of them — not a claim that they've been solved.

FOR ANYONE, NOT JUST ENGINEERS

Four roles, one loop that repeats.

No jargon required — this is the same system described on the previous pages, just traced through step by step.

SUPER ADMIN

Solution Tech Company Ltd's own account. Reviews and approves new businesses, records subscription payments, and oversees the whole platform.

BUSINESS

The shop or trader that signs up — the customer of KopaAlert. Tracks its own customers and debts, and can add staff.

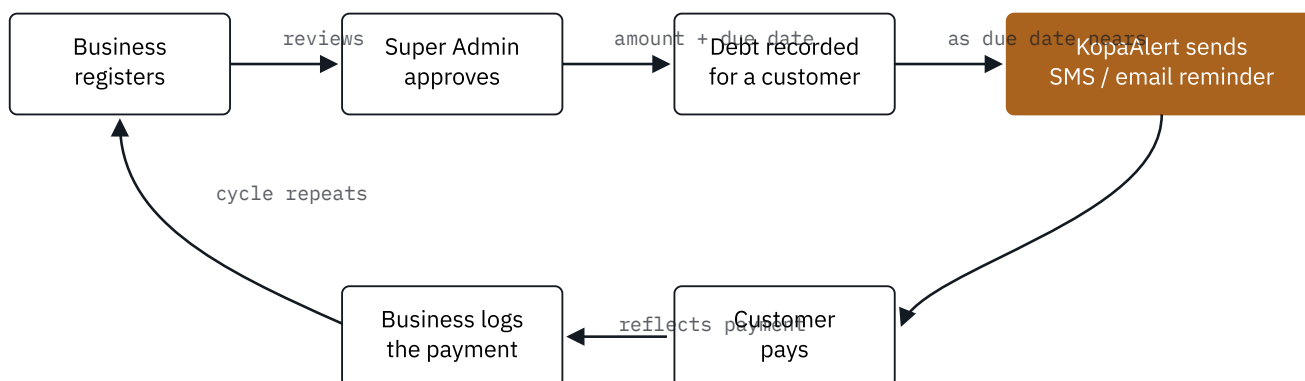
EMPLOYEE

Staff a business adds to its own account, with the same day-to-day access to customers and debts as the owner.

CUSTOMER

The person who owes the business money. Never logs in — only ever reached by SMS or email when a reminder goes out.

The loop



The core loop every business runs: register once, then repeat "record → remind → pay → log" for every customer, for as long as the subscription stays active.

Paying for it

A new business starts free. Beyond that, KopaAlert is paid for in one of two ways — a recurring monthly subscription, or a single one-time payment that removes the recurring bill for good. Either way, an invoice and receipt is emailed automatically the moment a payment is recorded — nothing is ever taken on trust without paperwork to back it up.

FOR DEVELOPERS

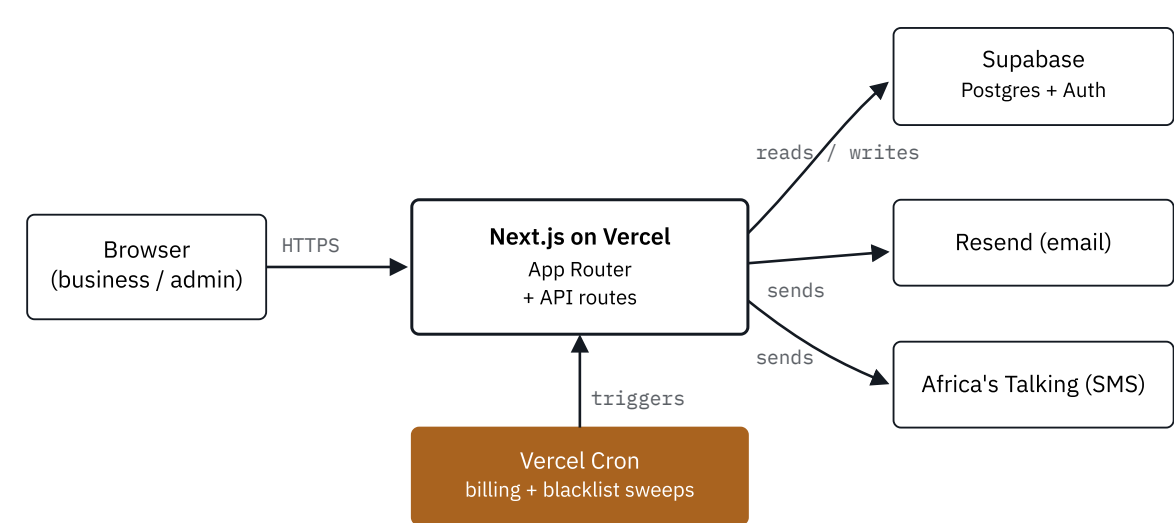
What it's actually built on, and where to start reading.

Written for a developer deciding whether to contribute to, maintain, or extend KopaAlert alongside — or eventually for — Solution Tech Company Ltd.

Stack

Framework	Next.js 15 (App Router) on React 19 — server components for data-loaded pages, client components for interactive forms.
Database & auth	Supabase (Postgres + built-in auth). Server and browser each get their own Supabase client via <code>@supabase/ssr</code> ; a service-role client handles admin-only writes.
Email	Resend, for every transactional email — invitations, approvals, invoices/receipts, renewal reminders and lock notices.
SMS	Africa's Talking, through one shared gateway account that KopaAlert operates centrally — no business configures its own SMS credentials.
Hosting	Vercel, including scheduled Cron Jobs for background billing and blacklist sweeps.
Styling	Tailwind CSS, with light/dark theme tokens throughout.

Architecture at a glance



Every request from the browser goes through the same Next.js app; the two background cron jobs re-enter that app's own API routes on a schedule rather than talking to Supabase directly.

Finding your way around

```
// where the actual logic lives, not just the routes src/app/ routes, grouped by who they're for admin/ super-admin-only pages (dashboard)/ business-facing pages (auth)/ login, register, password flows api/ route handlers – thin, delegate to src/lib src/lib/ the actual business logic admin/ e.g. create-super-admin.ts team/ e.g. invite-member.ts notifications/ email-templates.ts, resend.ts supabase/migrations/ hand-written SQL, applied via the CLI
```

A few early columns were added directly in the Supabase dashboard before being formally tracked as migrations — where that's true, it's noted in the migration file's own comments rather than hidden.

There's no public issue tracker or contribution guide yet — onboarding a collaborator is still a direct conversation. If you're a developer interested in maintaining, extending, or co-building the next phase of KopaAlert with Solution Tech Company Ltd, the contact on the closing page reaches Stanley directly.

DRAFT — FOR ANY BUSINESS ADOPTING KOPAAALERT

What a business gets, what it costs, and what happens if something goes wrong.

Written so nobody has to guess. This summarizes, in plain terms, what the live Terms of Service and Privacy Policy at kopaalert.shop already say in full — those remain the governing documents.

What every business gets

- An approved account with unlimited customer and debt records.
- Automated SMS and email reminders sent from KopaAlert's own shared gateway — no separate contract to set up.
- Payment recording, overdue flagging, and reporting on the dashboard.
- Role-based staff accounts, so an owner can add employees without sharing one login.
- An emailed invoice and receipt for every subscription payment, automatically, the moment it's recorded.

How it's paid for

MONTHLY SUBSCRIPTION

KES 1,500/mo

Default price, adjustable per business. Renews every 30 days from the last payment; a reminder is emailed 3 days before renewal.

ONE-TIME PAYMENT

Lifetime

A single payment removes the recurring bill permanently — no further renewals, no future invoices to chase.

If a payment lapses

Dashboard access is suspended, not deleted. Every customer, debt, and payment record stays exactly as it was, and access is restored immediately once payment is arranged — there is no penalty beyond the pause itself.

No guesswork

These terms apply to every business by default. Anything outside them — a custom price, a feature built specifically for one business, a dispute over an invoice — is settled directly, not guessed at from this document. Reach out using the contact on the closing page, or the support details already on every KopaAlert invitation and invoice email.

FOR WHOEVER MAINTAINS THIS NEXT

What's broken before, what's still open, and where it lives.

A running log, not a promise everything's clean — kept honest on purpose, for the same reason page three is.

Recurring pattern: dead-end links

The three issues below aren't three unrelated bugs — they're the same mistake surfacing in three different places. Each one is a link or action that looked correct in the component that rendered it, but led nowhere useful once the request reached the middleware: a page that should have opened stayed on login, or a redirect fired to the wrong destination, or a redirect fired at all when nothing should have happened. All three trace back to the same file for the same reason — route-level auth rules were extended one route at a time, and each new route had to be remembered rather than falling under a rule that covered it automatically.

01 **Terms & Privacy pages redirected signed-out visitors to /login** FIXED

The middleware's public-route allowlist never included `/terms` or `/privacy`, so anyone not signed in who clicked either link from the registration form or the cookie banner was bounced straight to the login page instead.

`src/lib/supabase/middleware.ts` — added both to a routes list that stays open regardless of auth state, separate from the routes that only make sense signed out.

02 **Signed-in super admin revisiting /admin/login just saw the form again** FIXED

`/admin/login` was exempt from the "send signed-in users to their dashboard" rule that `/login` and `/register` already had, so it did nothing for someone already authenticated.

`src/lib/supabase/middleware.ts` — now redirects a signed-in visitor to `/admin` (super admin) or `/dashboard` (anyone else).

03 **Password reset sent people to /login, which never actually showed** FIXED

Supabase's recovery flow leaves the user signed in once they set a new password, so pushing them to `/login` just meant middleware immediately redirected them onward to `/dashboard` anyway — harmless, but the code said one thing and did another.

`src/app/reset-password/page.tsx` – now pushes to `/dashboard` directly, matching what actually happens.

If a new page or link stops redirecting correctly, check `src/lib/supabase/middleware.ts` first – specifically whether the route is covered by the always-accessible list, the signed-out-only list, or neither. That's where all three of the above were fixed, and where the next one will be too.

Open

04 **Lint doesn't run in some environments** [OPEN](#)

`npm run lint` can fail with "Failed to load config next/core-web-vitals" where a machine-level ESLint config conflicts with this project's expectations and no `flat-config-eslint.config.js` exists yet in the repo. Type safety in the meantime comes from `npx tsc --noEmit`, which does run cleanly.

05 **Some businesses columns exist only as schema drift** [OPEN](#)

`business_code`, `subscription_tier`, `subscription_status`, `subscription_expires_at`, `sms_balance`, and the entire `subscription_payments` table were added by hand in the Supabase dashboard early on, before migrations tracked them. A fresh database built from the migration files alone won't have them. Documented in the header comment of `supabase/migrations/20260908000000_subscription_billing_schema.sql` – anyone provisioning a new environment needs to add those columns manually first.

06 **`src/types/database.types.ts` is hand-written and incomplete** [OPEN](#)

It's missing `Business`, `SubscriptionPayment`, and `PlatformSettings` entirely, unlike the usual Supabase-CLI-generated file. Nothing catches a typo'd column name on those tables at compile time.

Found something not listed here? This page is meant to grow – see the contact on the closing page.

CLOSING THIS CHAPTER, NOT THE PRODUCT

This dossier ends here. KopaAlert doesn't.

What's documented above is a real, live platform — running billing, reminders, and multi-tenant business accounts today at kopaalert.shop — built in spite of every constraint on page three, not after they were solved.

Closing this document is closing the first chapter of KopaAlert's build: from an idea that changed shape more than once, to a system a business can actually register for, get approved on, and run its debt collection through today. It is not a decision to stop building.

What's next



Direct M-Pesa payment (STK push) in place of admin-recorded payments.



A proper mobile experience for business owners on the move.



Deeper reporting and analytics per business.



Formal incorporation of Solution Tech Company Ltd.

Businesses considering KopaAlert, developers considering building alongside it, and anyone considering backing what Solution Tech Company Ltd becomes next — all of it starts with reaching out directly. No guesswork, on either side.

Contact

EMAIL

solutiontechcompany@gmail.com

PHONE

+254 740 305 253

PRODUCT

kopaalert.shop

— Stanley Murangiri M'mwirambua
Founder & Sole Developer, Solution Tech Company Ltd

